



React.js · Node.js · Express.js · MongoDB Atlas · JWT

Student Management System

Technical Project Report — MERN Stack

theechana54@gmail.com

<i>SUBMITTED_BY</i>	THEECHANA R
<i>PROJECT_TYPE</i>	Full-Stack Web Application
<i>REPOSITORY</i>	github.com/theechana54/Azure-VM-Website-Hosting
<i>LIVE_URL</i>	azure-vm-website-hosting-nine.vercel.app
<i>API_URL</i>	azure-vm-website-hosting.onrender.com

This report documents the technical design, architecture, and implementation of a full-stack Student Management System built using the MERN stack, including REST API design, authentication, database schema, and cloud deployment.

1. Objective

To develop a full-stack web application that enables educational institutions to efficiently manage student information. The project provides hands-on experience in frontend development, backend development, database management, REST APIs, user authentication, and cloud deployment using the MERN Stack (MongoDB, Express.js, React.js, Node.js).

2. Project Description

The Student Management System is a web application that allows administrators and users to digitally manage student records. It supports registering new students, updating student information, searching and filtering records, and deleting records. The application includes secure user authentication, role-based access control (Admin/User), and a fully responsive interface that works across desktop and mobile devices.

3. Technology Stack

Layer	Technology Used
Frontend	React.js, Tailwind CSS, Vite
Backend	Node.js, Express.js
Database	MongoDB Atlas
Authentication	JWT (JSON Web Token)
Frontend Hosting	Vercel
Backend Hosting	Render
Version Control	Git & GitHub
API Testing	Postman

4. Features

- User Registration & Login
- JWT-based Authentication
- Role-Based Access Control (Admin / User)
- Student Registration (Add new records)
- Update Student Details
- Delete Student Records
- Search & Filter Students (name, roll no, email, department)
- Dashboard with Student Statistics
- Responsive User Interface (Desktop & Mobile)
- Form Validation
- Export Student Records to Excel
- RESTful API Integration
- MongoDB Database Integration

5. Deployment

Backend (Render): The Express.js server is deployed as a Web Service on Render, connected directly to the GitHub repository, with environment variables configured for the MongoDB connection string and JWT secret.

Frontend (Vercel): The React application is deployed on Vercel, with the Root Directory set to the client folder and an environment variable pointing to the live backend API URL.

Database (MongoDB Atlas): A cloud-hosted MongoDB cluster stores all user and student data in the 'studentDB' database, accessed securely from the backend.

6.Data Schema (MongoDB Collections)

<i>Collection</i>	<i>Field</i>	<i>Type</i>	<i>Description</i>
users	name	String	Full name of the user
	email	String	Unique login email
	password	String	Hashed password (bcrypt)
	role	String	'admin' or 'user'
students	name	String	Student full name
	rollNo	String	Unique roll number
	department	String	Department name
	year	Number	Academic year
	email / phone	String	Contact details

7.REST API Endpoints

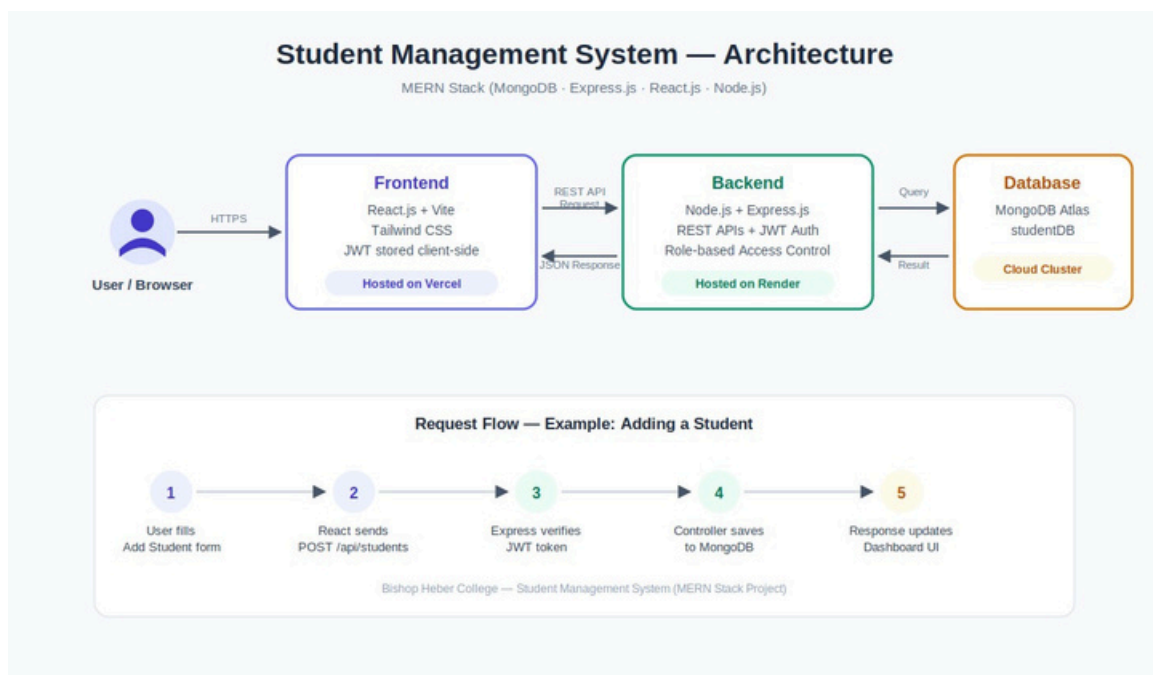
<i>Method</i>	<i>Endpoint</i>	<i>Description</i>	<i>Auth</i>
POST	/api/auth/register	Register a new user	No
POST	/api/auth/login	Login & receive JWT token	No
GET	/api/students	Get all students (search/filter)	Yes
POST	/api/students	Add a new student record	Yes
PUT	/api/students/:id	Update a student record	Yes
DELETE	/api/students/:id	Delete a student record	Yes
GET	/api/students/export	Export records to Excel	Yes

8. Project Folder Structure

```
Azure-VM-Website-Hosting/  
|-- client/                               React frontend (Vite)  
|   |-- src/pages/                        Register, Login, Dashboard  
|   |-- src/components/                  Reusable UI components  
|   |-- src/context/                     AuthContext (JWT state)  
|-- server/                               Node.js + Express backend  
|   |-- controllers/                     Route logic (auth, students)  
|   |-- models/                          Mongoose schemas  
|   |-- routes/                          API route definitions  
|   |-- middleware/                      JWT verification  
|   |-- server.js                        App entry point  
|-- README.md
```

9. System Architecture

The application follows a 3-tier client-server architecture. The React.js frontend (hosted on Vercel) communicates with the Node.js/Express.js backend (hosted on Render) through RESTful APIs over HTTPS. The backend authenticates requests using JWT tokens and performs CRUD operations on a MongoDB Atlas cloud database.



10. Learning Outcomes

- Building a complete MERN Stack application from scratch
- Designing and consuming RESTful APIs
- Performing CRUD (Create, Read, Update, Delete) operations
- Implementing Authentication and Role-Based Authorization
- Connecting a React frontend to a Node/Express backend
- Managing a cloud-hosted MongoDB database
- Deploying full-stack applications using Render and Vercel
- Using Git & GitHub for version control
- Debugging real-world deployment issues (CORS, environment variables, build settings)

11. Future Enhancements

- Add pagination for large student datasets
- Add profile picture upload for students
- Add email notifications for account creation
- Add an attendance tracking module
- Add data visualization (charts) on the dashboard
- Implement password reset via email

12. Conclusion

The Student Management System successfully demonstrates the complete lifecycle of a full-stack web application — from designing the database schema and building secure REST APIs, to developing a responsive user interface and deploying the application to the cloud. The project provided practical, real-world experience in the MERN stack and in resolving common deployment challenges, meeting all the objectives outlined for this course.